String Methods Demonstration in Java

1. Aim

To write and execute a Java program that demonstrates commonly used String methods such as length(), trim(), charAt(), substring(), case conversion, searching, comparison, replacement, and splitting.

2. Description

This program demonstrates 15 commonly used Java String methods. It creates a String containing leading and trailing spaces and applies different methods to inspect, modify, search, compare, and split the string. Arrays.toString() is used to display the array produced by split().

3. Algorithm

1. Start the program.

2. Import java.util.Arrays.

3. Create the string " Hello, Java World! ".

4. Apply length() and trim().

5. Use charAt() and substring() to access and extract characters.

6. Use toUpperCase() and toLowerCase().

7. Use contains(), indexOf(), and lastIndexOf() for searching.

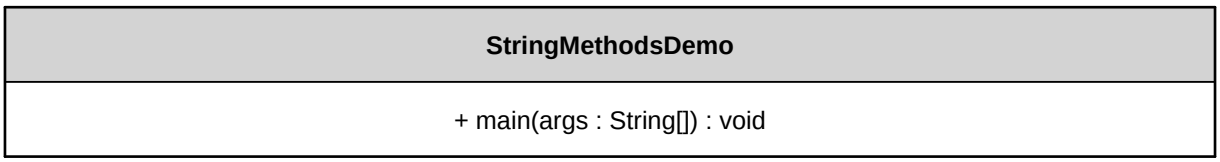
8. Use replace(), startsWith(), and endsWith().

9. Use equals() and equalsIgnoreCase() for comparison.

10. Use split() to divide the string into words.

11. Display all results and end the program.

4. UML Diagram



5. Source Code with Comments

```
package core_java;
import java.util.Arrays;

public class StringMethodsDemo {
    public static void main(String[] args) {
        String str = "  Hello, Java World!  ";
        System.out.println("Original String: '" + str + "'\n");

        // 1. length() - Returns the number of characters
        int length = str.length();
        System.out.println("1. length(): " + length);

        // 2. trim() - Removes leading and trailing whitespace
        String trimmed = str.trim();
        System.out.println("2. trim(): '" + trimmed + "'");

        // 3. charAt(index) - Returns the character at a specific position
        char character = trimmed.charAt(7);
        System.out.println("3. charAt(7): " + character);

        // 4. substring(beginIndex, endIndex) - Extracts a portion of the string
        String sub = trimmed.substring(7, 11);
        System.out.println("4. substring(7, 11): " + sub);

        // 5. toUpperCase() - Converts all characters to uppercase
        String upper = trimmed.toUpperCase();
        System.out.println("5. toUpperCase(): " + upper);

        // 6. toLowerCase() - Converts all characters to lowercase
        String lower = trimmed.toLowerCase();
```

```
        System.out.println("6. toLowerCase(): " + lower);

        // 7. contains(sequence) - Checks if a string contains a specific sequence
        boolean hasJava = trimmed.contains("Java");
        System.out.println("7. contains(\"Java\"): " + hasJava);

        // 8. replace(old, new) - Replaces characters or substrings
        String replaced = trimmed.replace("World", "Universe");
        System.out.println("8. replace(): " + replaced);

        // 9. indexOf(str) - Finds the index of the first occurrence of a substring
        int firstIndex = trimmed.indexOf("a");
        System.out.println("9. indexOf('a'): " + firstIndex);

        // 10. lastIndexOf(str) - Finds the index of the last occurrence of a substring
        int lastIndex = trimmed.lastIndexOf("a");
        System.out.println("10. lastIndexOf('a'): " + lastIndex);

        // 11. startsWith(prefix) - Checks if the string starts with a specific prefix
        boolean starts = trimmed.startsWith("Hello");
        System.out.println("11. startsWith(\"Hello\"): " + starts);

        // 12. endsWith(suffix) - Checks if the string ends with a specific suffix
        boolean ends = trimmed.endsWith("!");
        System.out.println("12. endsWith(\"!\"): " + ends);

        // 13. equals(anotherString) - Compares contents with case sensitivity
        boolean isEqual = trimmed.equals("hello, java world!");
        System.out.println("13. equals() [case-sensitive]: " + isEqual);

        // 14. equalsIgnoreCase(anotherString) - Compares contents ignoring case
        boolean isEqualIgnore = trimmed.equalsIgnoreCase("hello, java world!");
        System.out.println("14. equalsIgnoreCase(): " + isEqualIgnore);

        // 15. split(regex) - Splits the string into an array based on a matching delimiter
        String[] words = trimmed.split(" ");
        System.out.println("15. split(\" \"): " + Arrays.toString(words));
    }
}
```

6. Compilation and Execution Commands

```
javac -d . StringMethodsDemo.java
java core_java.StringMethodsDemo
```

7. Output Screenshot

```
$ javac -d . StringMethodsDemo.java
$ java core_java.StringMethodsDemo

Original String: '  Hello, Java World!  '

1. length(): 21
2. trim(): 'Hello, Java World!'
3. charAt(7): J
4. substring(7, 11): Java
5. toUpperCase(): HELLO, JAVA WORLD!
6. toLowerCase(): hello, java world!
7. contains("Java"): true
8. replace(): Hello, Java Universe!
9. indexOf('a'): 8
10. lastIndexOf('a'): 10
11. startsWith("Hello"): true
12. endsWith("!"): true
13. equals() [case-sensitive]: false
14. equalsIgnoreCase(): true
15. split(" "): [Hello,, Java, World!]
```

8. Result

The program was successfully compiled and executed. The output demonstrates the behavior and results of 15 commonly used Java String methods.